BOOTCAMP III / RELATÓRIO TÉCNICO

Governança, segurança e ética no uso de IA

Projeto TaskAnalyzer | Entrega Final (Fase 3) | 25/10/2026

1. Objetivo e contexto

A entrega final consolida um TaskAnalyzer especificado na Fase 1, implementado e testado conforme a Fase 2 e preparado para integração contínua e containerização na Fase 3. O uso de IA foi assistivo: geração de código e documentação com contrato explícito, testes automatizados e necessidade de revisão humana.

2. Rastreabilidade: contrato original e implementação

Regra da Fase 1	Evidência implementada
id_tarefa: inteiro ≥ 0; prioridade baixa/média/alta	Validação em src/task_analyzer.py e testes parametrizados.
datas UTC; data_inicio e prazo obrigatórios	Validação de ISO 8601 UTC, datas timezone-aware e campos obrigatórios.
status: concluída, pendente, cancelada	Enumeração fechada; pendentes/canceladas fora de duração e atraso.
duração em minutos desde data_inicio	Média geral e por prioridade sobre concluídas; duas casas decimais.
atraso somente de concluídas após o prazo	Comparação data_conclusao > prazo; caso de igualdade testado.
resultado com quatro campos no contrato inicial	analyze_tasks mantém as quatro chaves; resumo_fase2 é separado.

3. Riscos de alucinação e controles

Risco na geração por IA	Mitigação / teste
Confundir data_criacao e data_inicio no cálculo	Cenário específico altera criação e comprova que a duração permanece igual.
Trocar minutos por horas sem autorização	Interface principal expressa minutos; adaptador Fase 2 faz conversão explícita.
Omitir status cancelada ou retirar acentos	Enums literais da Fase 1 e testes com status/prioridades incorretos.
Inventar bibliotecas, banco ou persistência	Somente biblioteca padrão para aplicação; teste verifica ausência de arquivos gravados.
Ignorar datas inválidas sem transparência	TaskValidationError identifica tarefa/campo; CLI retorna código 2.

4. Tratamento de divergências entre especificações

Datas inconsistentes: §1.3.3 da Fase 1 orienta ignorar registros inconsistentes, mas o Cenário 2 da mesma fase determina exceção específica e a Fase 2 exige TaskValidationError. Como decisão fundamentada, a validação rejeita entradas inválidas e não calcula agregados parciais. A escolha fica explícita em CONTEXT_RULES.md.

Unidades e campos: a Fase 1 define tempo_medio_conclusao_min e somente quatro chaves; a Fase 2 usa horas e totais por status. A compatibilidade é exposta em resumo_fase2, mantendo a assinatura original de analyze_tasks.

Denominador do atraso: a Fase 1 define percentual de concluídas após o prazo, sem escrever a fórmula do denominador. A implementação usa concluídas atrasadas / concluídas x 100. Canceladas e pendentes não entram no denominador; zeros retornam 0.0.

Quantidade de tarefas: são consideradas todas as tarefas válidas (inclusive pendentes e canceladas), enquanto tempo e atraso usam concluídas. São decisões interpretativas documentadas, sujeitas à validação do professor.

5. Human-in-the-loop: revisão e homologação

O código não é considerado automaticamente correto apenas por ter sido gerado com IA. A revisão compreende a comparação do contrato com a Fase 1 original, leitura crítica das funções, checagem manual do cenário de cinco tarefas, execução da suíte, análise de tratamentos de erro e confirmação dos registros reais do CI e do Docker antes da apresentação.

6. Evidências verificadas e pendentes

Verificação	Situação confirmada em 25/10/2026
Pytest local	65 casos aprovados (execução local deste projeto).
Cenário válido	5 tarefas, 3 concluídas, média 80 min, 1 atraso / 3 concluídas = 33,33%.
Cenário inválido	Conclusão anterior ao início gera TaskValidationError e saída 2 na CLI.
GitHub Actions	Workflow criado; execução remota verde deve ser confirmada no GitHub.
Docker Compose	Dockerfile e compose preparados; execução em contêiner ainda não confirmada neste ambiente.
Pull Request	Não foi criado nesta elaboração; revisão/merge deve ocorrer no repositório real.

7. Ética, segurança e sustentabilidade

O módulo trabalha em memória, não envia dados a APIs externas, não armazena tarefas e não grava dados pessoais. O logging registra somente contadores agregados. A dependência de desenvolvimento é fixada em requirements.txt; a aplicação utiliza bibliotecas da linguagem. A infraestrutura de CI/Docker visa consistência e reduz diferenças entre ambientes, mas sua execução real é condição de evidência.

8. Conclusão

A solução integra contrato SDD, validação explícita, 65 testes locais, integração contínua declarada, containerização e análise ética. O resultado dos testes valida os comportamentos cobertos, não comprova cobertura integral nem execução remota. A homologação final exige observar os jobs e contêiner em execução e confirmar a aderência das interpretações de negócio à especificação do curso.